

DAILY TASK - 11

NAME : SANJEEV N

DATE : 19-08-2025

1. ADD USER

POST `/users/add` Add User

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "Id": 0,
  "Username": "Quiely",
  "Email": "Quiely@gmail.com",
  "Password": "Quiely"
}
```

2. VALIDATE USER

GET `/users/{username}/songs` Get User Songs

Parameters

Name	Description
username * required	
string (path)	Rocky

200

Response body

```
[
  {
    "Id": 1,
    "Title": "Perfect",
    "Artist": "Ed Sheeran",
    "Album": "Cold",
    "Year": 2020
  },
  {
    "Id": 2,
    "Title": "Bad Guy",
    "Artist": "Billie Eilish",
    "Album": "Rock",
    "Year": 2019
  },
  {
    "Id": 3,
    "Title": "Meesaya Murukku",
    "Artist": "Hip Hop Tamizha",
    "Album": "MM1",
    "Year": 2022
  }
]
```

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker, Session, declarative_base
from fastapi import FastAPI, Depends, HTTPException, Request
from typing import List
import uvicorn
from pydantic import BaseModel

SQLALCHEMY_DATABASE_URL = "sqlite:///./songs.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
Base = declarative_base()

class Song(Base):
    __tablename__ = 'songs'
    Id = Column(Integer, primary_key=True, index=True)
    Title = Column(String(100), nullable=False)
    Artist = Column(String(100), nullable=False)
```

```

    Album = Column(String(100), nullable=True)
    Year = Column(Integer, nullable=True)

class User(Base):
    __tablename__ = "users"
    Id = Column(Integer, primary_key=True, index=True)
    Username = Column(String(50), unique=True, nullable=False)
    Email = Column(String(100), unique=True, nullable=False)
    Password = Column(String(100), nullable=False)

Base.metadata.create_all(bind=engine)

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

class SongSchema(BaseModel):
    Id: int | None = None
    Title: str
    Artist: str
    Album: str | None = None
    Year: int | None = None

    class Config:
        orm_mode = True

class UserSchema(BaseModel):
    Id: int | None = None
    Username: str
    Email: str
    Password: str
    class Config:
        orm_mode = True

@app.post("/songs/add", response_model=SongSchema)

```

```

def add_song(song: SongSchema, db: Session = Depends(get_db)):
    s = Song(
        Title=song.Title,
        Artist=song.Artist,
        Album=song.Album,
        Year=song.Year
    )
    db.add(s)
    db.commit()
    db.refresh(s)
    return s

@app.get("/songs", response_model=List[SongSchema])
def list_songs(db: Session = Depends(get_db)):
    all_songs = db.query(Song).all()
    return all_songs

@app.get("/songs/{song_id}", response_model=SongSchema)
def get_song(song_id: int, db: Session = Depends(get_db)):
    s = db.query(Song).filter(Song.Id == song_id).first()
    if not s:
        return {"error": "Song not found"}
    return s

@app.post("/users/add", response_model=UserSchema)
def add_user(user: UserSchema, db: Session = Depends(get_db)):
    existing = db.query(User).filter(
        (User.Username == user.Username) | (User.Email == user.Email)
    ).first()
    if existing:
        raise HTTPException(status_code=400, detail="User already exists")

    u = User(
        Username=user.Username,
        Email=user.Email,
        Password=user.Password
    )
    db.add(u)

```

```

        db.commit()
        db.refresh(u)
        return u

@app.get("/users/{username}/songs", response_model=List[SongSchema])
def get_user_songs(username: str, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.Username == username).first()
    if not user:
        raise HTTPException(status_code=403, detail="User not found.
Access denied.")
    return db.query(Song).all()

@app.middleware("http")
async def addmiddleware(request: Request, call_next):
    print("Middleware Intercepted")
    response = await call_next(request)
    return response

if __name__ == "__main__":
    uvicorn.run("task:app", host="127.0.0.1", port=8000, reload=True)

```